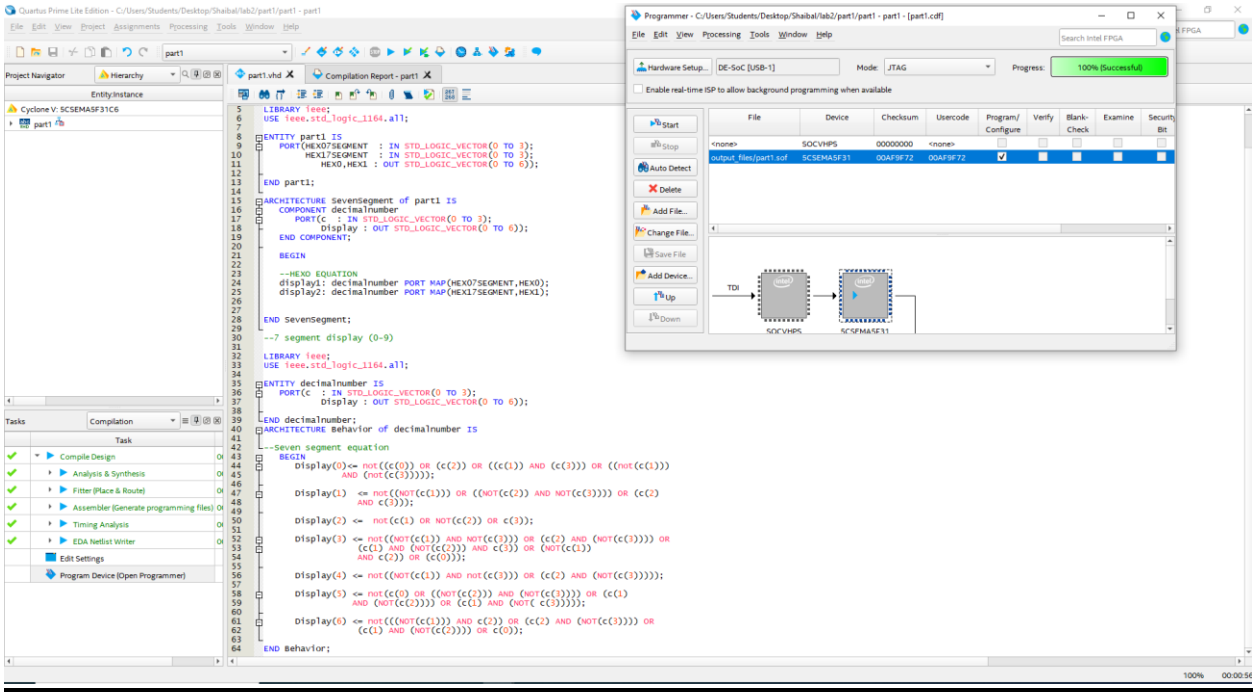
Objective: The purpose of this exercise is to learn in designing combinational circuits that can perform binary-to-decimal number conversions and binary-coded-decimal (BCD) addition.

- **Part1:** This is a basic program where we show 7-segment display output in two different 7 segment display in the DE-soc1 board. The code screenshot and a sample output what I get are given below:



```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY part1 IS
   PORT(HEX07SEGMENT  : IN STD_LOGIC_VECTOR(0 TO 3);
        HEX17SEGMENT  : IN STD_LOGIC_VECTOR(0 TO 3);
          HEX0,HEX1  : OUT STD_LOGIC_VECTOR(0 TO 6));

END part1;

ARCHITECTURE SevenSegment of part1 IS
   COMPONENT decimalnumber
      PORT(c  : IN STD_LOGIC_VECTOR(0 TO 3);
           Display : OUT STD_LOGIC_VECTOR(0 TO 6));
   END COMPONENT;

   BEGIN

   --HEX0 EQUATION
   display1: decimalnumber PORT MAP(HEX07SEGMENT,HEX0);
   display2: decimalnumber PORT MAP(HEX17SEGMENT,HEX1);

END SevenSegment;

--7 segment display (0-9)

LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY decimalnumber IS
   PORT(c  : IN STD_LOGIC_VECTOR(0 TO 3);
           Display : OUT STD_LOGIC_VECTOR(0 TO 6));
END decimalnumber;
ARCHITECTURE Behavior of decimalnumber IS

--Seven segment equation
   BEGIN
      Display(0)<= not((c(0)) OR (c(2)) OR (((c(1)) AND (c(3))) OR ((not(c(1)))
                 AND (not(c(3))))));

      Display(1)  <= not((NOT(c(1))) OR ((NOT(c(2)) AND NOT(c(3)))) OR (c(2)
                  AND c(3)));

      Display(2) <=  not(c(1) OR NOT(c(2)) OR c(3));

      Display(3) <= not((NOT(c(1)) AND NOT(c(3))) OR (c(2) AND (NOT(c(3)))) OR
                  (c(1) AND (NOT(c(2))) AND c(3)) OR (NOT(c(1))
                  AND c(2)) OR (c(0)));

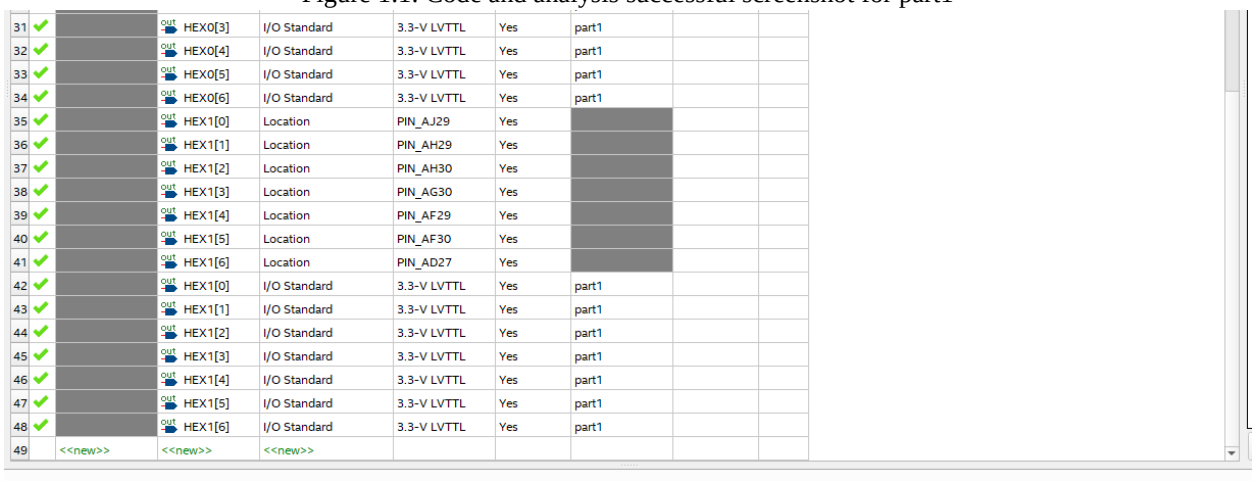
      Display(4) <= not((NOT(c(1)) AND not(c(3))) OR (c(2) AND (NOT(c(3)))));

      Display(5) <= not(c(0) OR ((NOT(c(2))) AND (NOT(c(3)))) OR (c(1)
                 AND (NOT(c(2)))) OR (c(1) AND (NOT( c(3)))));

      Display(6) <= not(((NOT(c(1))) AND c(2)) OR (c(2) AND (NOT(c(3)))) OR
                  (c(1) AND (NOT(c(2)))) OR c(0));

END Behavior;
```

Figure 1.1: Code and analysis successful screenshot for part1



31	✓		HEX0[3]	I/O Standard	3.3-V LVTTL	Yes	part1	
32	✓		HEX0[4]	I/O Standard	3.3-V LVTTL	Yes	part1	
33	✓		HEX0[5]	I/O Standard	3.3-V LVTTL	Yes	part1	
34	✓		HEX0[6]	I/O Standard	3.3-V LVTTL	Yes	part1	
35	✓		HEX1[0]	Location	PIN_AJ29	Yes		
36	✓		HEX1[1]	Location	PIN_AH29	Yes		
37	✓		HEX1[2]	Location	PIN_AH30	Yes		
38	✓		HEX1[3]	Location	PIN_AG30	Yes		
39	✓		HEX1[4]	Location	PIN_AF29	Yes		
40	✓		HEX1[5]	Location	PIN_AF30	Yes		
41	✓		HEX1[6]	Location	PIN_AD27	Yes		
42	✓		HEX1[0]	I/O Standard	3.3-V LVTTL	Yes	part1	
43	✓		HEX1[1]	I/O Standard	3.3-V LVTTL	Yes	part1	
44	✓		HEX1[2]	I/O Standard	3.3-V LVTTL	Yes	part1	
45	✓		HEX1[3]	I/O Standard	3.3-V LVTTL	Yes	part1	
46	✓		HEX1[4]	I/O Standard	3.3-V LVTTL	Yes	part1	
47	✓		HEX1[5]	I/O Standard	3.3-V LVTTL	Yes	part1	
48	✓		HEX1[6]	I/O Standard	3.3-V LVTTL	Yes	part1	
49		<<new>>	<<new>>	<<new>>				

Figure1.2: Pin assignment of part1

part1.vhd* | Compilation Report - part1 | Assignment Editor

<<new>> | Filter on node names: *

	Status	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
1	✓		HEX0...T[0]	Location	PIN_AB12	Yes			
2	✓		HEX0...T[1]	Location	PIN_AC12	Yes			
3	✓		HEX0...T[2]	Location	PIN_AF9	Yes			
4	✓		HEX0...T[3]	Location	PIN_AF10	Yes			
5	✓		HEX1...T[0]	Location	PIN_AD11	Yes			
6	✓		HEX1...T[1]	Location	PIN_AD12	Yes			
7	✓		HEX1...T[2]	Location	PIN_AE11	Yes			
8	✓		HEX1...T[3]	Location	PIN_AC9	Yes			
9	?		SW[8]	Location	PIN_AD10	Yes			
10	?		SW[9]	Location	PIN_AE12	Yes			
11	?		SW[0]	I/O Standard	3.3-V LVTTL	Yes	part1		
12	?		SW[1]	I/O Standard	3.3-V LVTTL	Yes	part1		
13	?		SW[2]	I/O Standard	3.3-V LVTTL	Yes	part1		
14	?		SW[3]	I/O Standard	3.3-V LVTTL	Yes	part1		
15	?		SW[4]	I/O Standard	3.3-V LVTTL	Yes	part1		
16	?		SW[5]	I/O Standard	3.3-V LVTTL	Yes	part1		
17	?		SW[6]	I/O Standard	3.3-V LVTTL	Yes	part1		
18	?		SW[7]	I/O Standard	3.3-V LVTTL	Yes	part1		
19	?		SW[8]	I/O Standard	3.3-V LVTTL	Yes	part1		
20	?		SW[9]	I/O Standard	3.3-V LVTTL	Yes	part1		
21	✓		HEX0[0]	Location	PIN_AE26	Yes			
22	✓		HEX0[1]	Location	PIN_AE27	Yes			
23	✓		HEX0[2]	Location	PIN_AE28	Yes			
24	✓		HEX0[3]	Location	PIN_AG27	Yes			
25	✓		HEX0[4]	Location	PIN_AF28	Yes			
26	✓		HEX0[5]	Location	PIN_AG28	Yes			
27	✓		HEX0[6]	Location	PIN_AH28	Yes			
28	✓		HEX0[0]	I/O Standard	3.3-V LVTTL	Yes	part1		

Figure1.3: pin assignment of part1

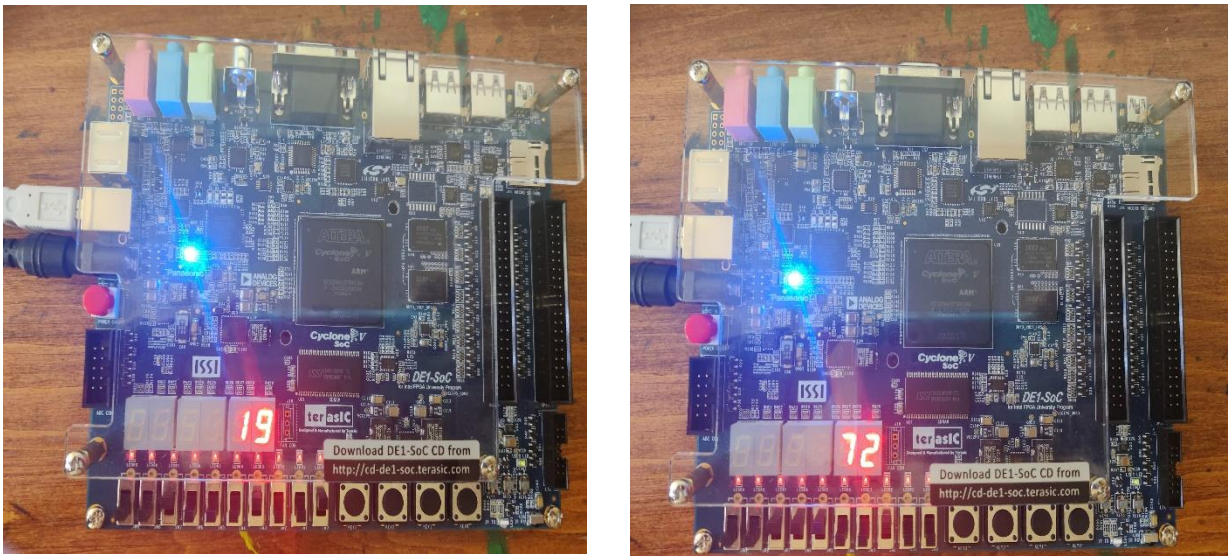


Figure1.4: Sample output of part1

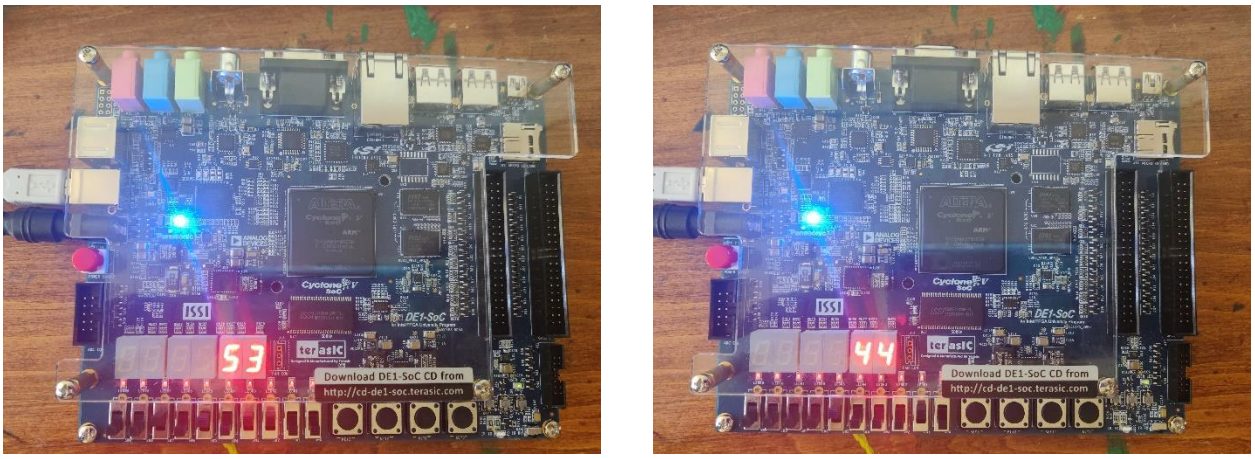


Figure1.5: Sample output of part1 (2)

- **Part2:** In part 2, I need to design a circuit that converts a four-bit binary number into its two-digit decimal equivalent. Furthermore, I need to show it in 7 segment display. I need to design a comparator (>9) means carry 1 and circuit A for change value like for ("10" in binary we get 1010, but in 7-segment we need 0000 for hex0 and 0001 for hex1). Basically, I did binary to decimal conversion. The code screenshot and a sample output what I get are given below:

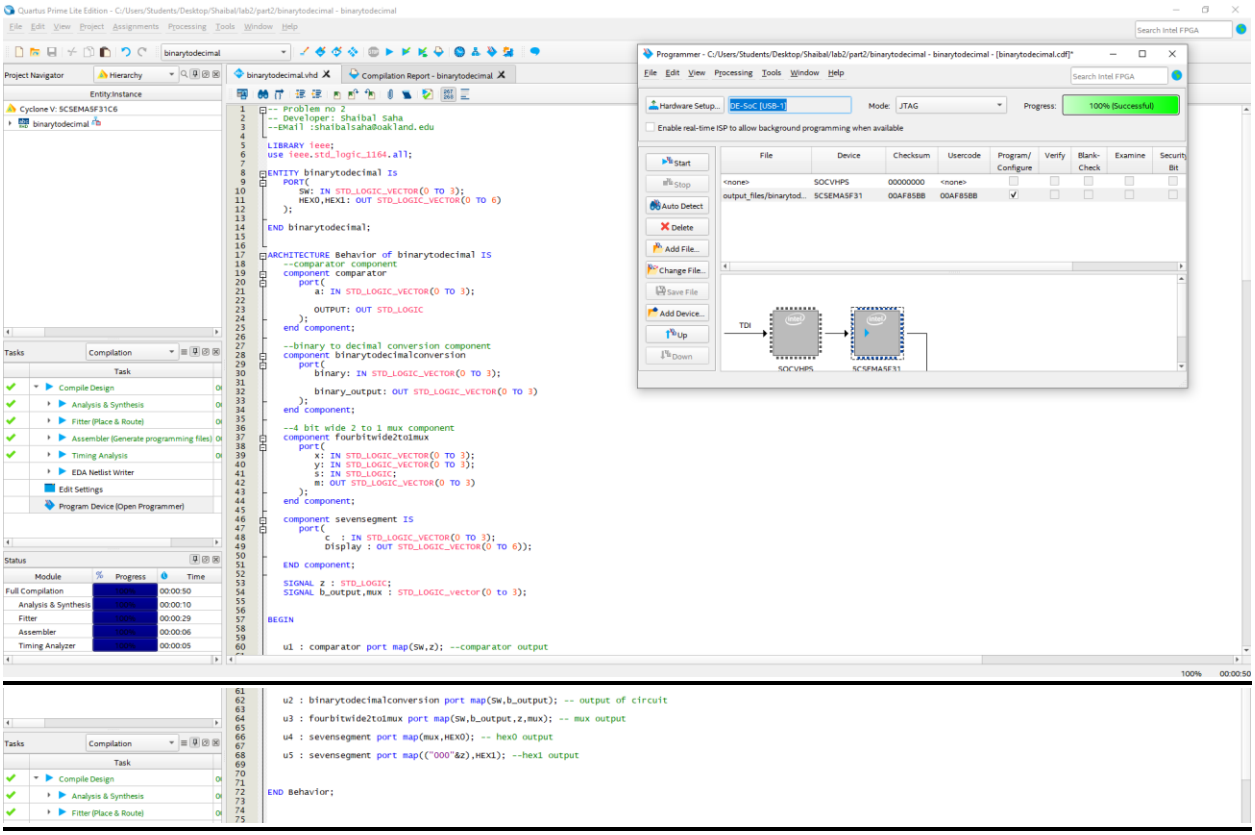


Figure 2.1: Code and analysis successful screenshot for part2

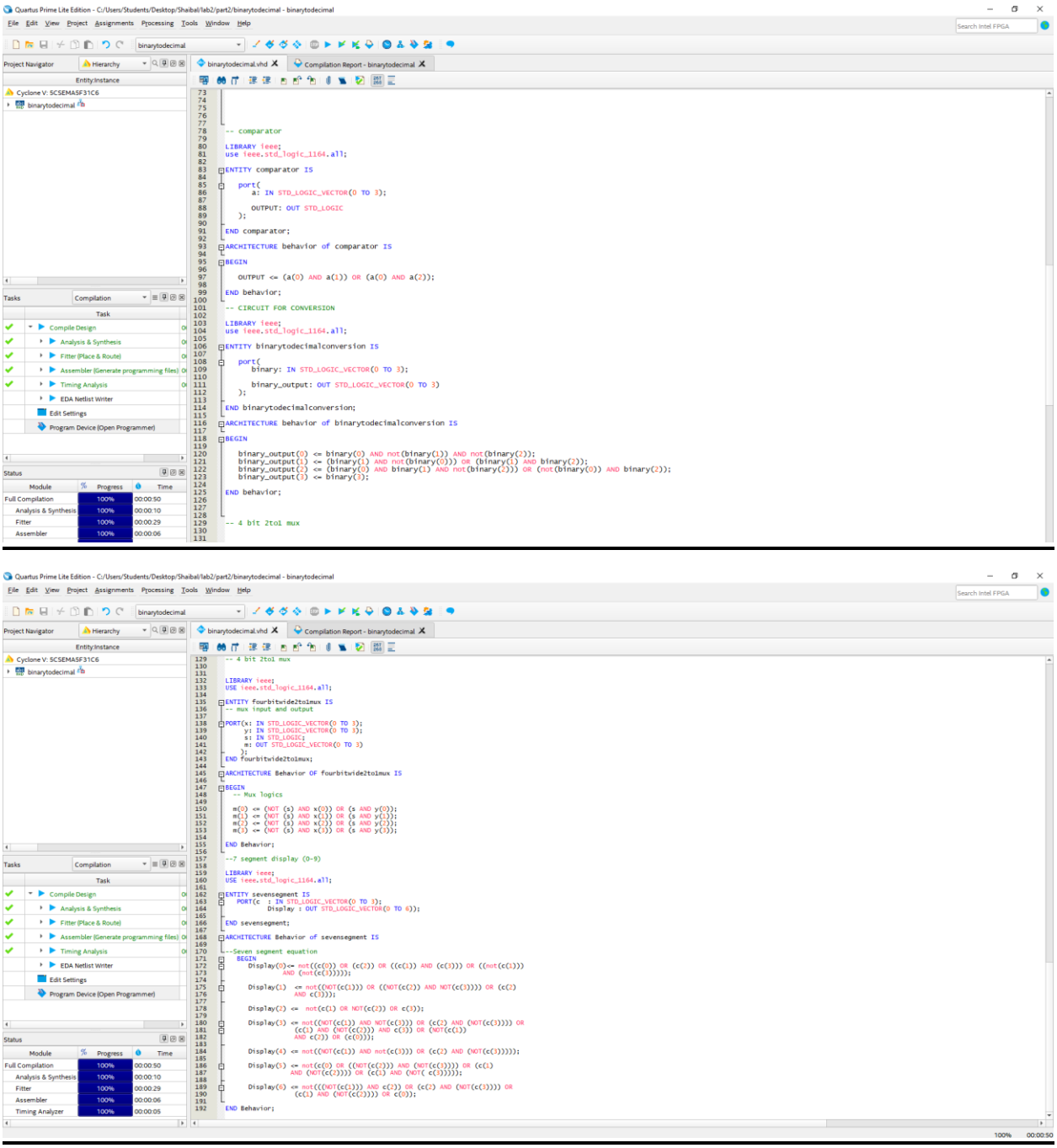


Figure 2.2: Code and analysis successful screenshot for part2

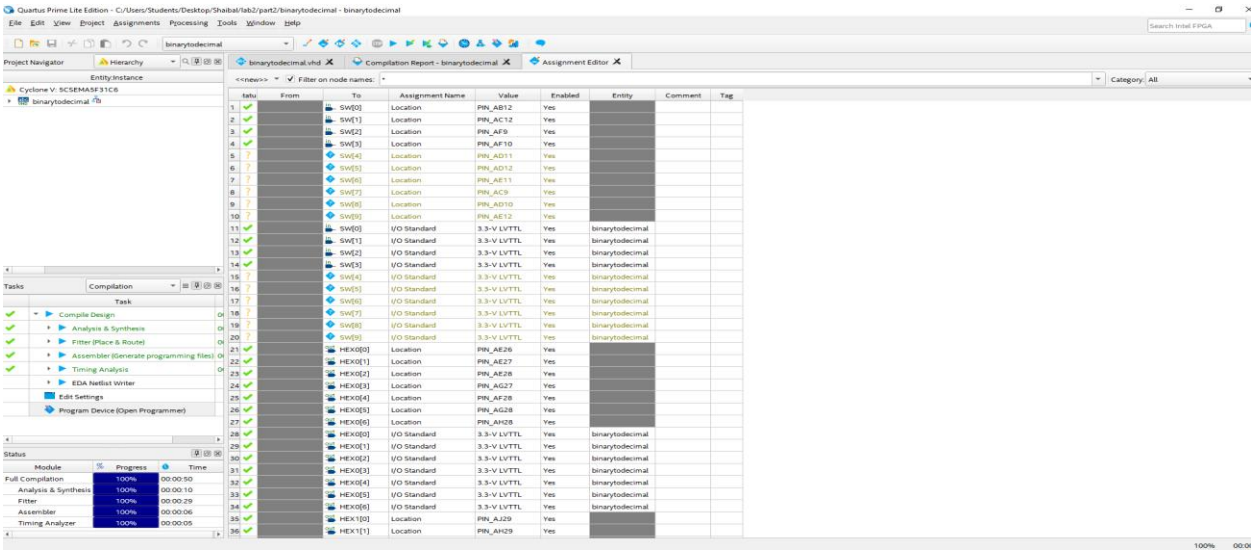


Figure 2.3: Pin assignment screenshot for part2

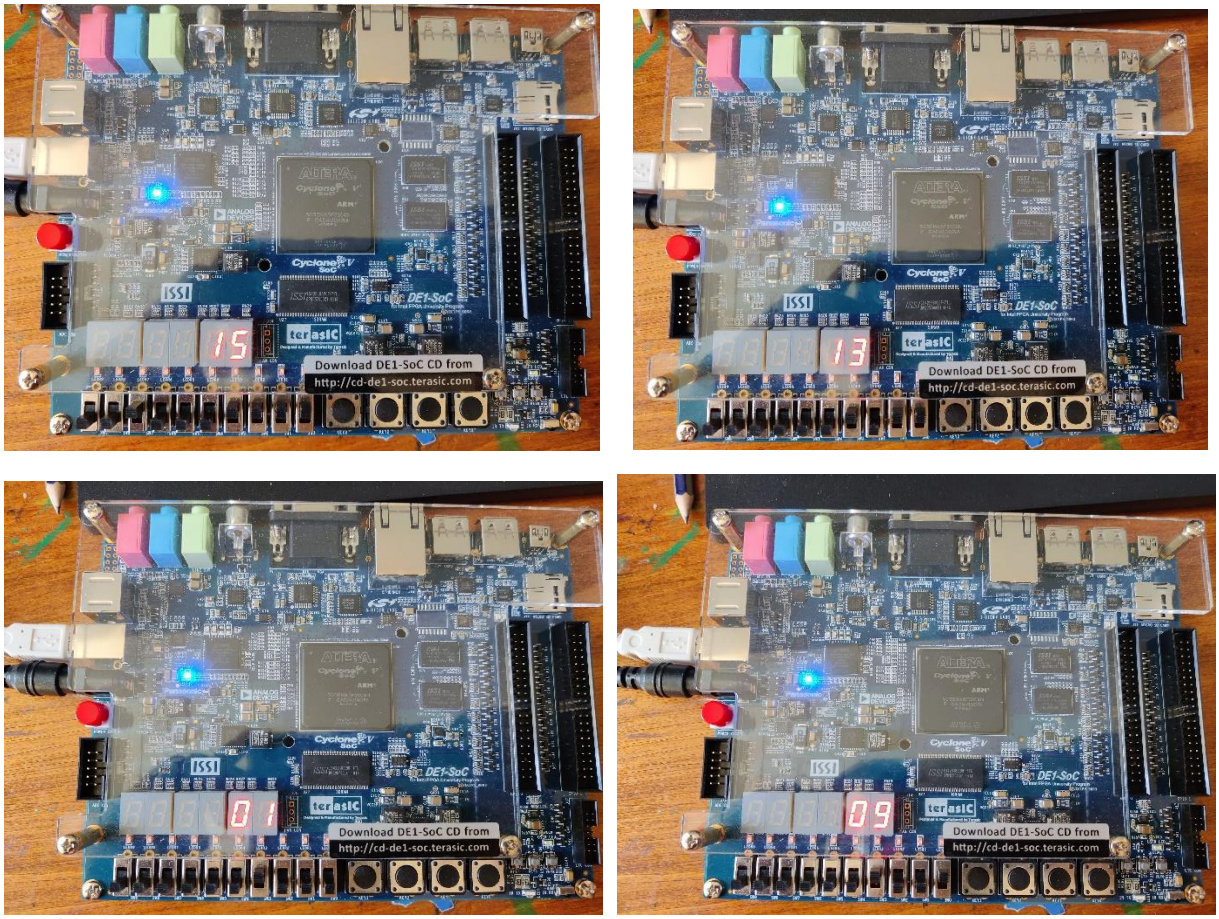


Figure 2.4: Sample output for part2

- **Part3:** This is the VHDL code for ripple carry adder where the full adder uses the carry output of the previous full adder as a carry input and gives out of sum for individual full adder as well as a final carryout. In this design, I show carry out in LEDR (9). And sum in LEDR (0 to 3). The code design screenshot and a sample output what I get are given below:

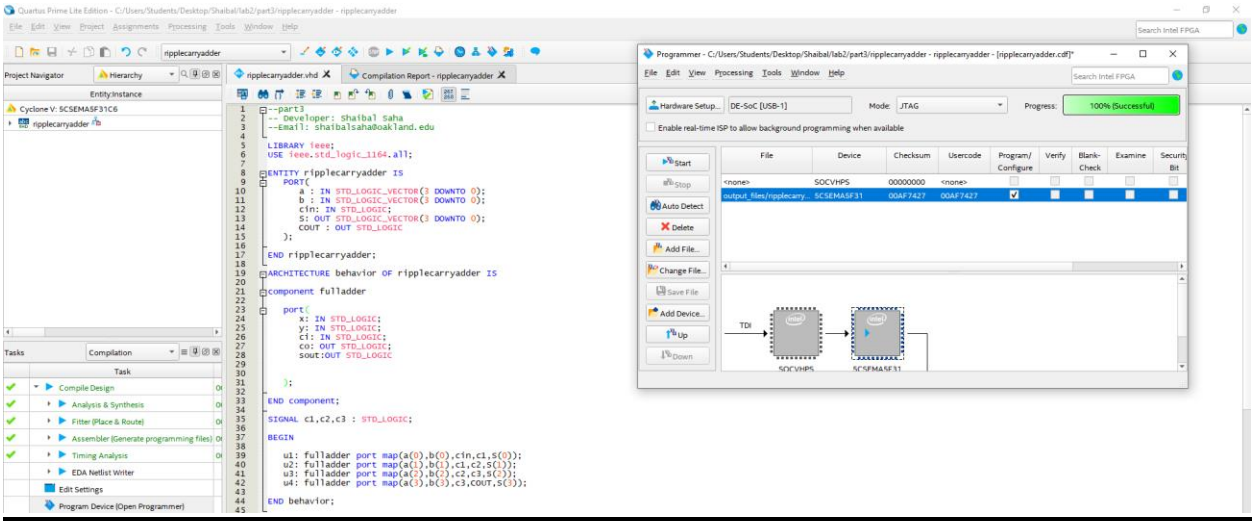


Figure 3.1: code and analysis screenshot for part3

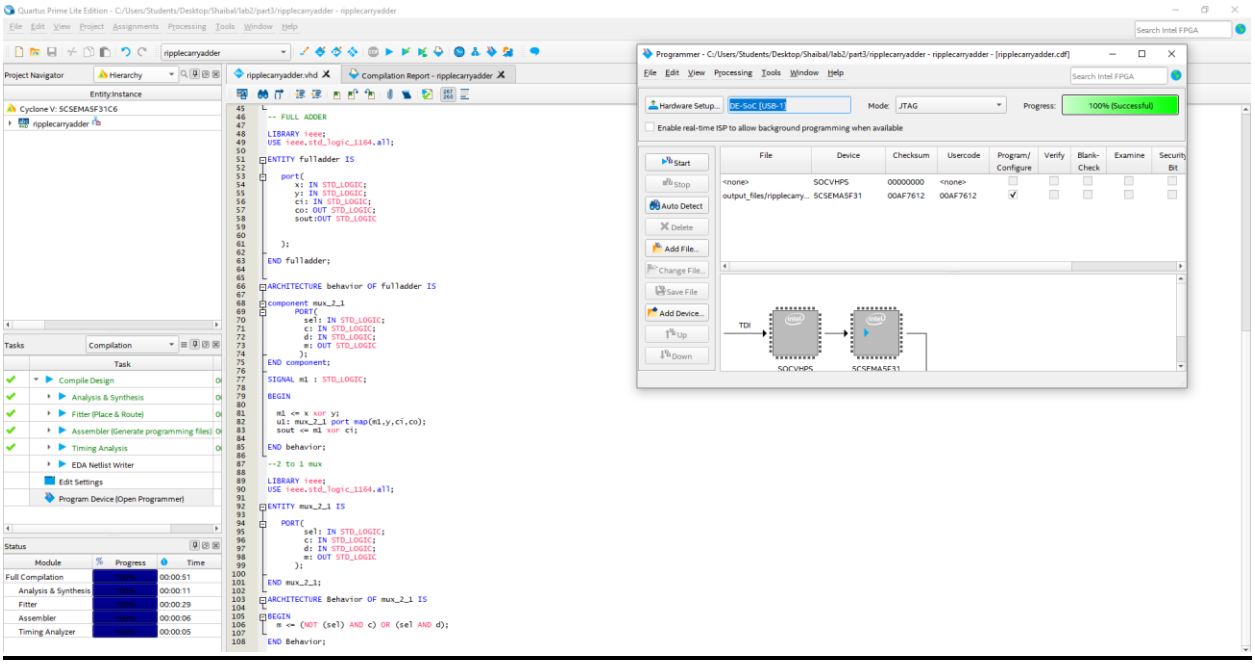
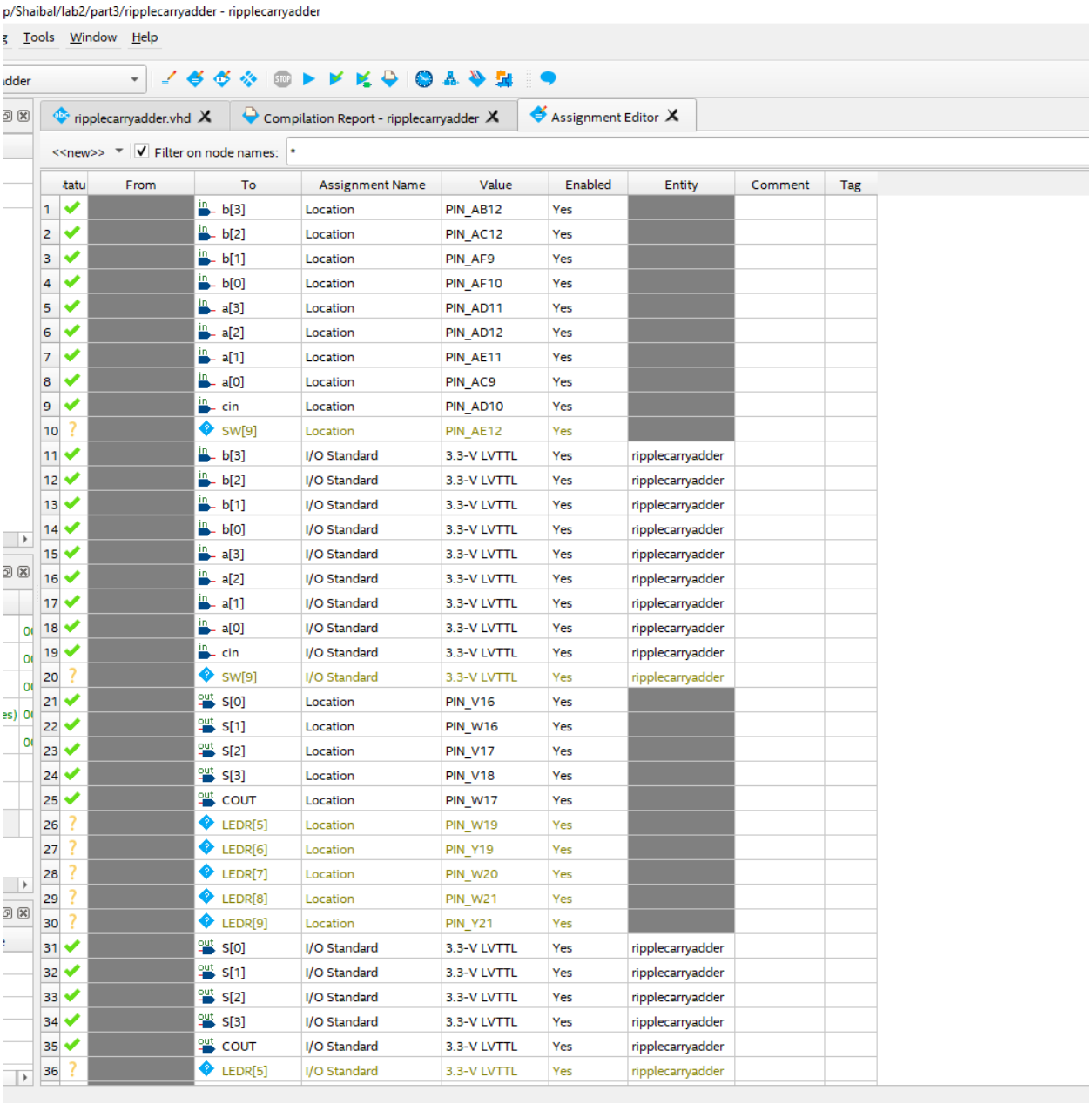


Figure 3.2: code and analysis screenshot for part3



	tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
1	✔		b[3]	Location	PIN_AB12	Yes			
2	✔		b[2]	Location	PIN_AC12	Yes			
3	✔		b[1]	Location	PIN_AF9	Yes			
4	✔		b[0]	Location	PIN_AF10	Yes			
5	✔		a[3]	Location	PIN_AD11	Yes			
6	✔		a[2]	Location	PIN_AD12	Yes			
7	✔		a[1]	Location	PIN_AE11	Yes			
8	✔		a[0]	Location	PIN_AC9	Yes			
9	✔		cin	Location	PIN_AD10	Yes			
10	?		SW[9]	Location	PIN_AE12	Yes			
11	✔		b[3]	I/O Standard	3.3-V LVTTL	Yes	ripplecarryadder		
12	✔		b[2]	I/O Standard	3.3-V LVTTL	Yes	ripplecarryadder		
13	✔		b[1]	I/O Standard	3.3-V LVTTL	Yes	ripplecarryadder		
14	✔		b[0]	I/O Standard	3.3-V LVTTL	Yes	ripplecarryadder		
15	✔		a[3]	I/O Standard	3.3-V LVTTL	Yes	ripplecarryadder		
16	✔		a[2]	I/O Standard	3.3-V LVTTL	Yes	ripplecarryadder		
17	✔		a[1]	I/O Standard	3.3-V LVTTL	Yes	ripplecarryadder		
18	✔		a[0]	I/O Standard	3.3-V LVTTL	Yes	ripplecarryadder		
19	✔		cin	I/O Standard	3.3-V LVTTL	Yes	ripplecarryadder		
20	?		SW[9]	I/O Standard	3.3-V LVTTL	Yes	ripplecarryadder		
21	✔		S[0]	Location	PIN_V16	Yes			
22	✔		S[1]	Location	PIN_W16	Yes			
23	✔		S[2]	Location	PIN_V17	Yes			
24	✔		S[3]	Location	PIN_V18	Yes			
25	✔		COUT	Location	PIN_W17	Yes			
26	?		LEDR[5]	Location	PIN_W19	Yes			
27	?		LEDR[6]	Location	PIN_Y19	Yes			
28	?		LEDR[7]	Location	PIN_W20	Yes			
29	?		LEDR[8]	Location	PIN_W21	Yes			
30	?		LEDR[9]	Location	PIN_Y21	Yes			
31	✔		S[0]	I/O Standard	3.3-V LVTTL	Yes	ripplecarryadder		
32	✔		S[1]	I/O Standard	3.3-V LVTTL	Yes	ripplecarryadder		
33	✔		S[2]	I/O Standard	3.3-V LVTTL	Yes	ripplecarryadder		
34	✔		S[3]	I/O Standard	3.3-V LVTTL	Yes	ripplecarryadder		
35	✔		COUT	I/O Standard	3.3-V LVTTL	Yes	ripplecarryadder		
36	?		LEDR[5]	I/O Standard	3.3-V LVTTL	Yes	ripplecarryadder		

Figure 3.3: Pin assignment for part3

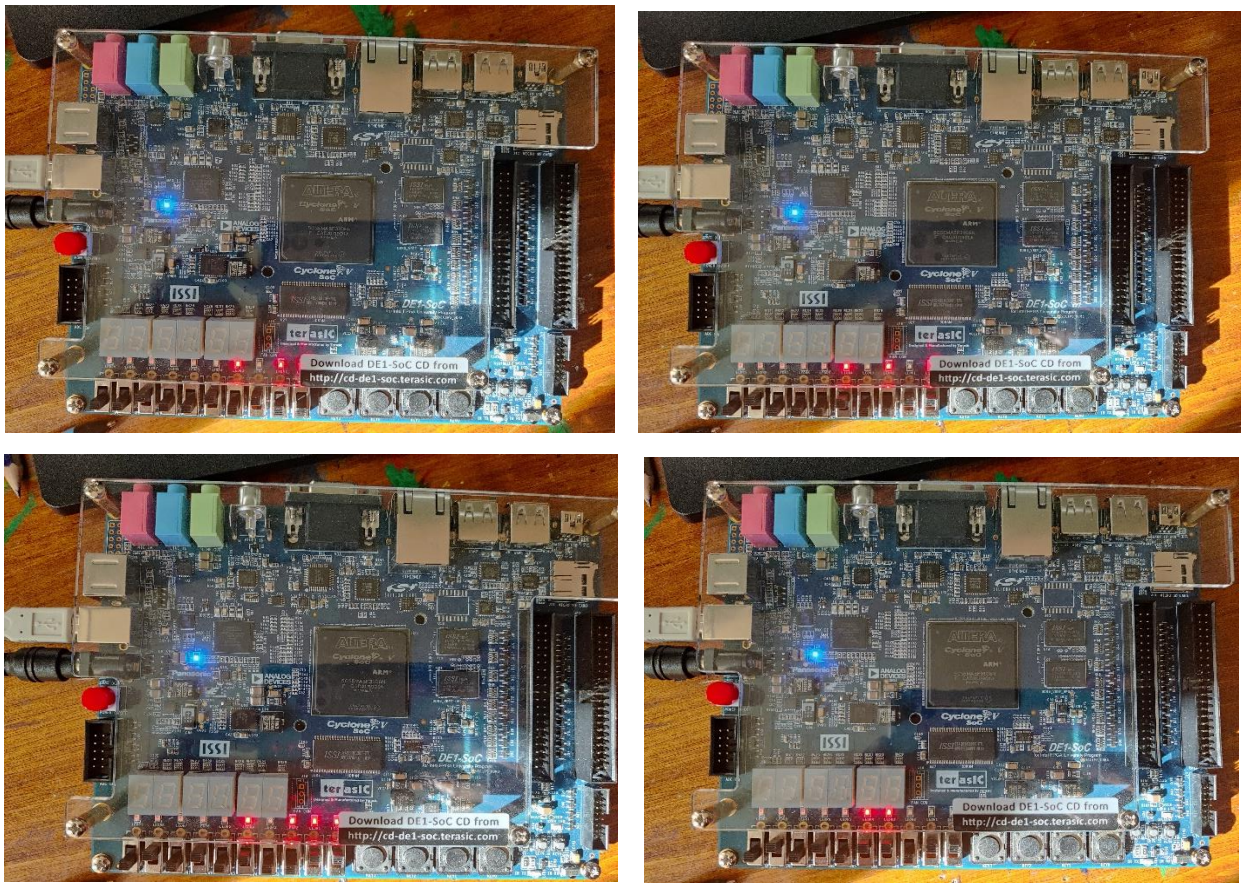


Figure 3.4: Sample output for part3

- **Part4:** This is the VHDL code for summation of two decimal and show it in 7 segments. In this part, as 7 segments can only show 9 highest. Highest sum we can get (9+9+1=19) where 1 is the carry input. I use the part 2 and part 3 portion and modified the circuit A from the part 2 for getting sum from 16 to 19. I also asked to show a LEDR for error if any user do input of greater than 9('1001'). HEX5 and HEX3 to represent 4 bits input each and HEX1 and HEX0 shows the output. LEDR9 shows the error output. The code design screenshot and a sample output what I get are given below:

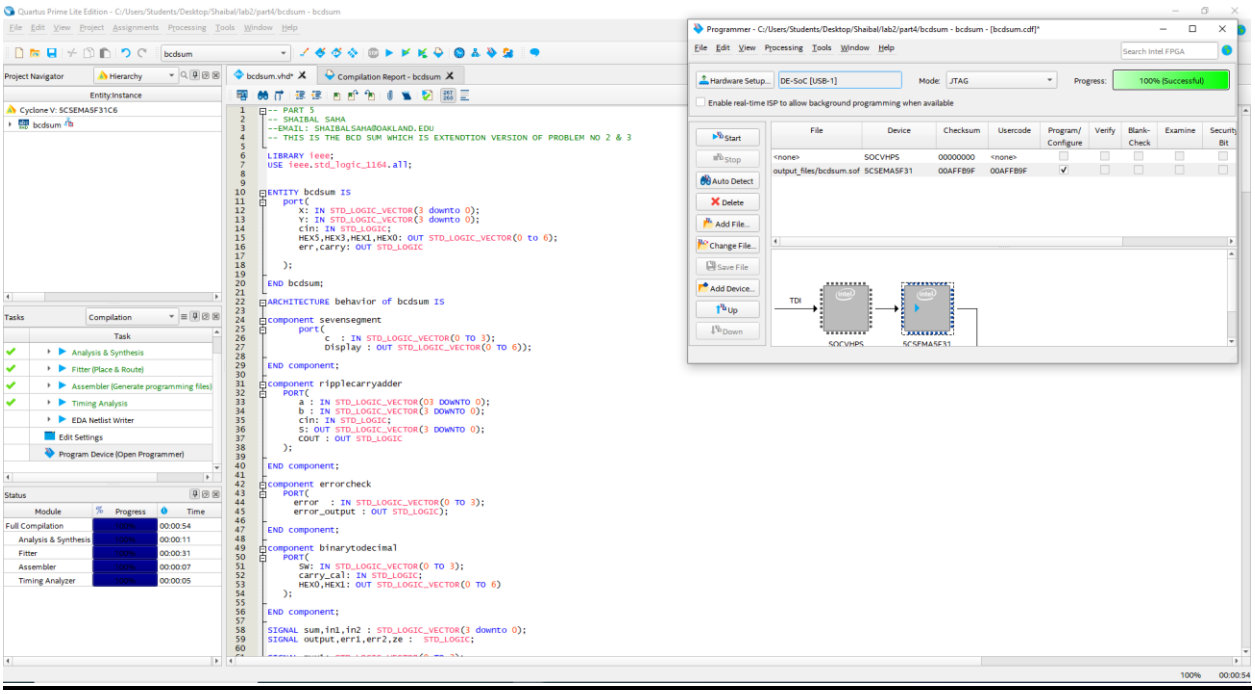


Figure 4.1 code and analysis screenshot for part4(main body function)

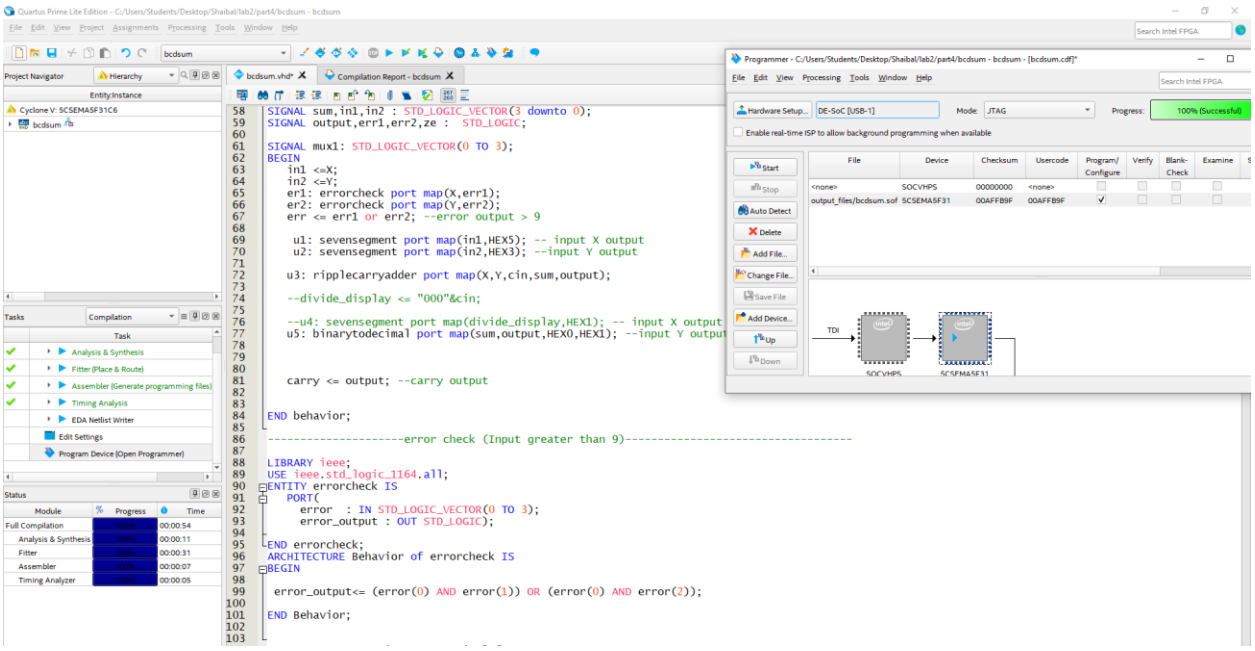


Figure 4.2: code and analysis screenshot for part4 (display output and error check)

```
----------------Binary TO DEcimal from part 2---------------------------------

LIBRARY ieee;
use ieee.std_logic_1164.all;

ENTITY binarytodecimal Is
   PORT(
      SW: IN STD_LOGIC_VECTOR(0 TO 3);
      carry_cal: IN STD_LOGIC;
      HEX0,HEX1: OUT STD_LOGIC_VECTOR(0 TO 6)
   );
END binarytodecimal;


ARCHITECTURE Behavior of binarytodecimal IS
   --comparator component
   component comparator
      port(
         a: IN STD_LOGIC_VECTOR(0 TO 3);

         OUTPUT: OUT STD_LOGIC
      );
   end component;

   --binary to decimal conversion component
   component binarytodecimalconversion
      port(
         binary: IN STD_LOGIC_VECTOR(0 TO 3);
         carry_bit :  IN STD_LOGIC;
         binary_output: OUT STD_LOGIC_VECTOR(0 TO 3)
      );
   end component;

   --4 bit wide 2 to 1 mux component
   component fourbitwide2to1mux
      port(
         x: IN STD_LOGIC_VECTOR(0 TO 3);
         y: IN STD_LOGIC_VECTOR(0 TO 3);
         s: IN STD_LOGIC;
         m: OUT STD_LOGIC_VECTOR(0 TO 3)
      );
   end component;

   component sevensegment IS
      port(
           c  : IN STD_LOGIC_VECTOR(0 TO 3);
           Display : OUT STD_LOGIC_VECTOR(0 TO 6));

   END component;

   SIGNAL z, znew : STD_LOGIC;
   SIGNAL b_output,mux : STD_LOGIC_vector(0 to 3);
```

Figure 4.3: code and analysis screenshot for part4 (binary to decimal from part 2)

```
BEGIN


   u1 : comparator port map(SW,z); --comparator output
   znew <= z or carry_cal;

   u2 : binarytodecimalconversion port map(SW,carry_cal,b_output); -- output of circuit

   u3 : fourbitwide2to1mux port map(SW,b_output,znew,mux); -- mux output

   u4 : sevensegment port map(mux,HEX0); -- hex0 output


   u5 : sevensegment port map(("000"&znew),HEX1); --hex1 output


END Behavior;




--------------------------------comparator---------------------------------------------

LIBRARY ieee;
use ieee.std_logic_1164.all;

ENTITY comparator IS

   port(
      a: IN STD_LOGIC_VECTOR(0 TO 3);

      OUTPUT: OUT STD_LOGIC
   );

END comparator;

ARCHITECTURE behavior of comparator IS

BEGIN

   OUTPUT <= (a(0) AND a(1)) OR (a(0) AND a(2));

END behavior;
```

Figure 4.4: code and analysis screenshot for part4 (continue part of figure 4.3 and comparator from part 2)

```
270   --------------------7 segment display (0-9)----------------------------------
271
272   LIBRARY ieee;
273   USE ieee.std_logic_1164.all;
274
275   ENTITY sevensegment IS
276      PORT(c   : IN STD_LOGIC_VECTOR(0 TO 3);
277               Display : OUT STD_LOGIC_VECTOR(0 TO 6));
278   END sevensegment;
279
280   ARCHITECTURE Behavior of sevensegment IS
281
282   --Seven segment equation
283      BEGIN
284         Display(0)<= not((c(0)) OR (c(2)) OR ((c(1)) AND (c(3))) OR ((not(c(1)))
285                     AND (not(c(3)))));
286
287         Display(1)  <= not((NOT(c(1))) OR ((NOT(c(2)) AND NOT(c(3)))) OR (c(2)
288                        AND c(3)));
289
290         Display(2) <=  not(c(1) OR NOT(c(2)) OR c(3));
291
292         Display(3) <= not((NOT(c(1)) AND NOT(c(3))) OR (c(2) AND (NOT(c(3)))) OR
293                       (c(1) AND (NOT(c(2))) AND c(3)) OR (NOT(c(1))
294                       AND c(2)) OR (c(0)));
295
296         Display(4) <= not((NOT(c(1)) AND not(c(3))) OR (c(2) AND (NOT(c(3)))));
297
298         Display(5) <= not(c(0) OR ((NOT(c(2))) AND (NOT(c(3)))) OR (c(1)
299                     AND (NOT(c(2)))) OR (c(1) AND (NOT( c(3)))));
300
301         Display(6) <= not(((NOT(c(1))) AND c(2)) OR (c(2) AND (NOT(c(3)))) OR
302                       (c(1) AND (NOT(c(2)))) OR c(0));
303
304   END Behavior;
305
306
307
```

Figure 4.5: code and analysis screenshot for part4 (7-segment display from part 2)

```
309   ------------------------------RIPPLE CARRY ADDER---------------------------------
310
311   LIBRARY ieee;
312   USE ieee.std_logic_1164.all;
313
314   ENTITY ripplecarryadder IS
315      PORT(
316         a : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
317         b : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
318         cin: IN STD_LOGIC;
319         S: OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
320         COUT : OUT STD_LOGIC
321      );
322   END ripplecarryadder;
323
324   ARCHITECTURE behavior OF ripplecarryadder IS
325
326   component fulladder
327
328      port(
329         x: IN STD_LOGIC;
330         y: IN STD_LOGIC;
331         ci: IN STD_LOGIC;
332         co: OUT STD_LOGIC;
333         sout:OUT STD_LOGIC
334
335
336      );
337   END component;
338
339   SIGNAL c1,c2,c3 : STD_LOGIC;
340
341   BEGIN
342
343      u1: fulladder port map(a(0),b(0),cin,c1,S(0));
344      u2: fulladder port map(a(1),b(1),c1,c2,S(1));
345      u3: fulladder port map(a(2),b(2),c2,c3,S(2));
346      u4: fulladder port map(a(3),b(3),c3,COUT,S(3));
347
348   END behavior;
349
350
351
352
```

Figure 4.6: code and analysis screenshot for part4 Ripple carry adder from part 3)

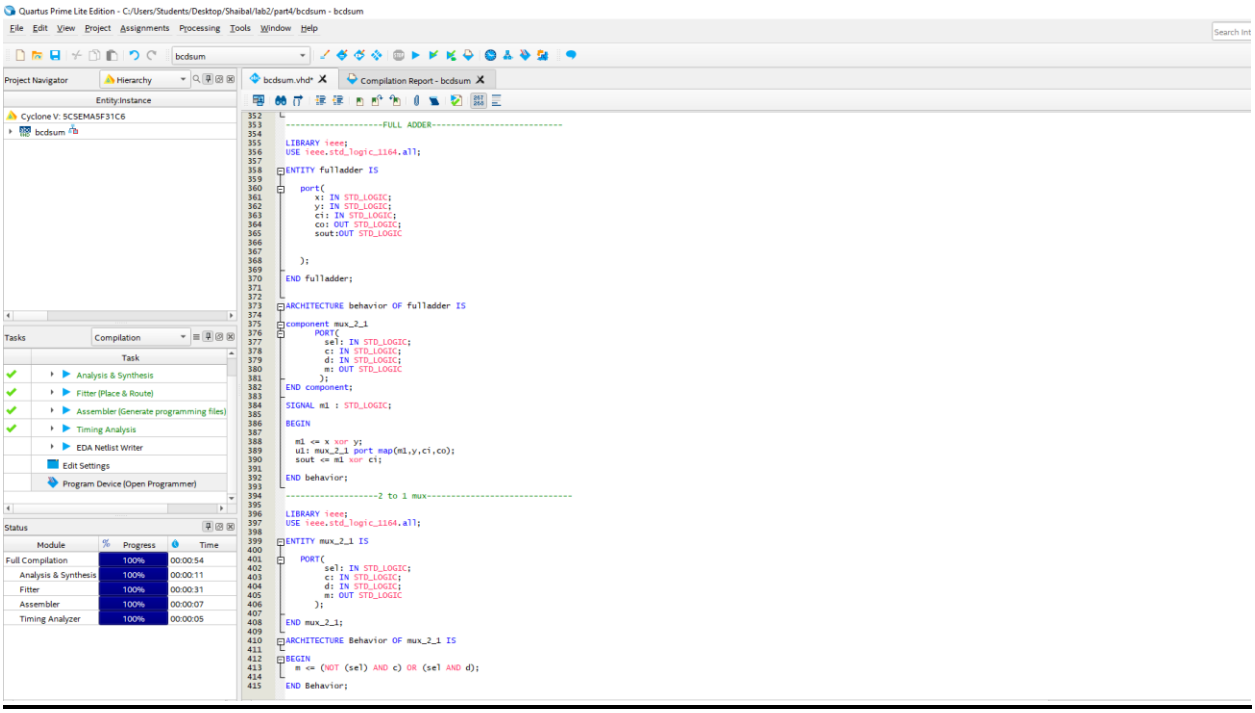
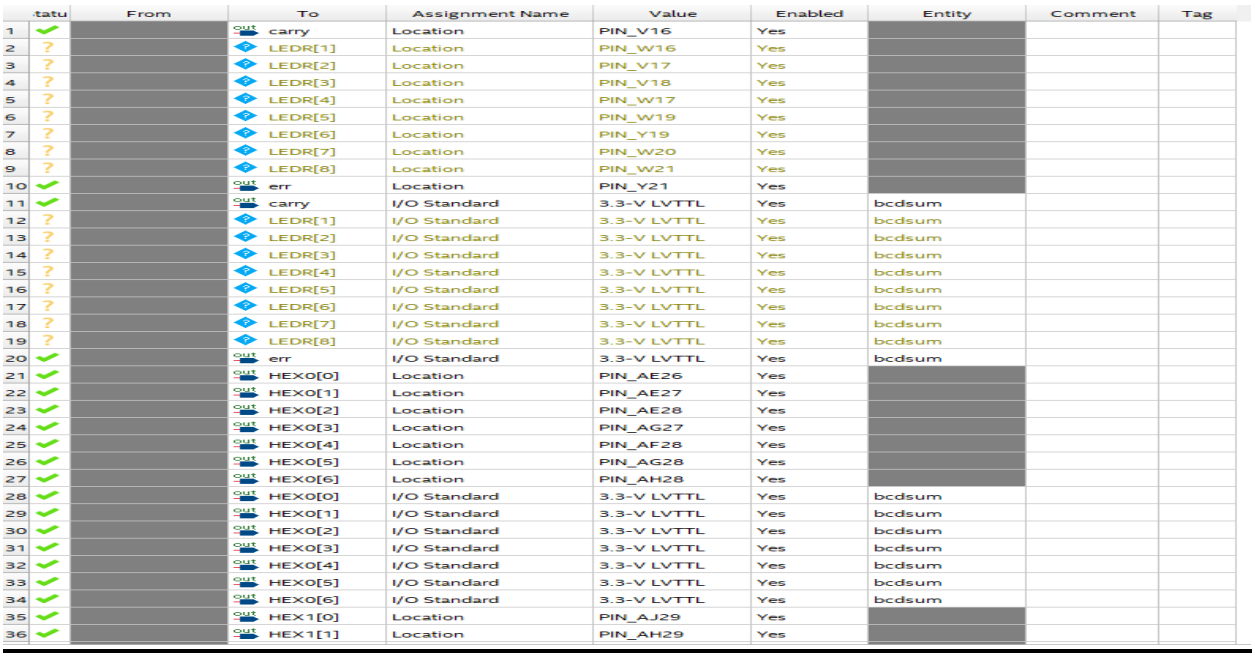
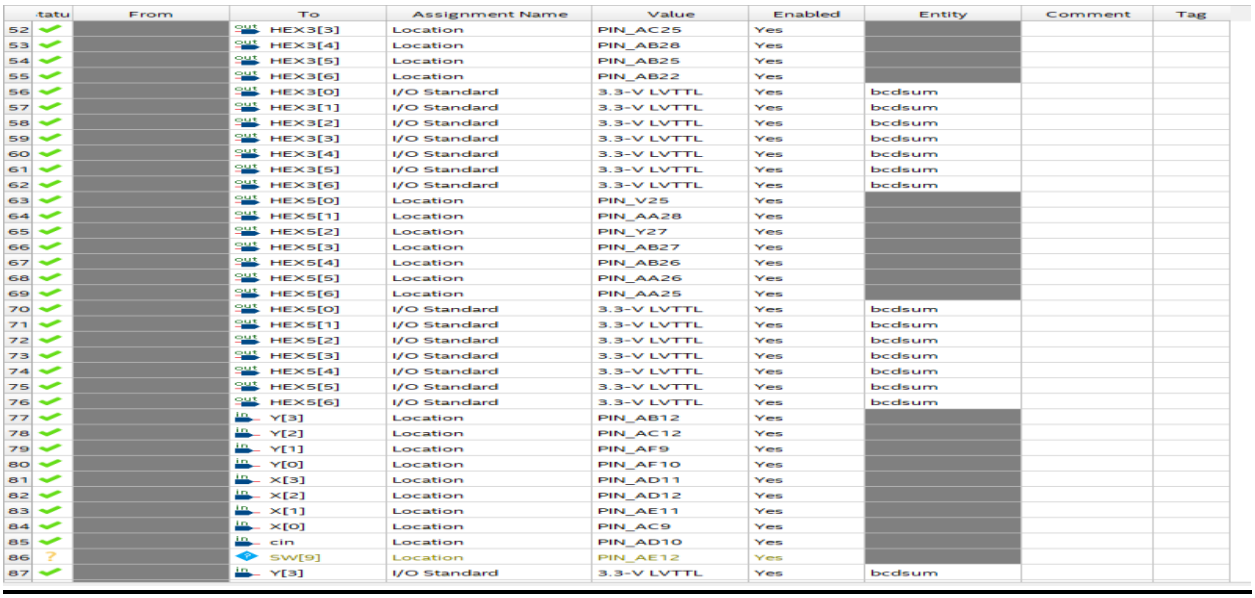


Figure 4.7: code and analysis screenshot for part4 (full adder from part 3)



	tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
1	✓		carry	Location	PIN_V16	Yes			
2	?		LEDR[1]	Location	PIN_W16	Yes			
3	?		LEDR[2]	Location	PIN_V17	Yes			
4	?		LEDR[3]	Location	PIN_V18	Yes			
5	?		LEDR[4]	Location	PIN_W17	Yes			
6	?		LEDR[5]	Location	PIN_W19	Yes			
7	?		LEDR[6]	Location	PIN_Y19	Yes			
8	?		LEDR[7]	Location	PIN_W20	Yes			
9	?		LEDR[8]	Location	PIN_W21	Yes			
10	✓		err	Location	PIN_Y21	Yes			
11	✓		carry	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
12	?		LEDR[1]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
13	?		LEDR[2]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
14	?		LEDR[3]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
15	?		LEDR[4]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
16	?		LEDR[5]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
17	?		LEDR[6]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
18	?		LEDR[7]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
19	?		LEDR[8]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
20	✓		err	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
21	✓		HEX0[0]	Location	PIN_AE26	Yes			
22	✓		HEX0[1]	Location	PIN_AE27	Yes			
23	✓		HEX0[2]	Location	PIN_AE28	Yes			
24	✓		HEX0[3]	Location	PIN_AG27	Yes			
25	✓		HEX0[4]	Location	PIN_AF28	Yes			
26	✓		HEX0[5]	Location	PIN_AG28	Yes			
27	✓		HEX0[6]	Location	PIN_AH28	Yes			
28	✓		HEX0[0]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
29	✓		HEX0[1]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
30	✓		HEX0[2]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
31	✓		HEX0[3]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
32	✓		HEX0[4]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
33	✓		HEX0[5]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
34	✓		HEX0[6]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
35	✓		HEX1[0]	Location	PIN_AJ29	Yes			
36	✓		HEX1[1]	Location	PIN_AH29	Yes			

Figure 4.8: Pin assignment of part4



	tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
52	✓		HEX3[3]	Location	PIN_AC25	Yes			
53	✓		HEX3[4]	Location	PIN_AB28	Yes			
54	✓		HEX3[5]	Location	PIN_AB25	Yes			
55	✓		HEX3[6]	Location	PIN_AB22	Yes			
56	✓		HEX3[0]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
57	✓		HEX3[1]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
58	✓		HEX3[2]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
59	✓		HEX3[3]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
60	✓		HEX3[4]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
61	✓		HEX3[5]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
62	✓		HEX3[6]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
63	✓		HEX5[0]	Location	PIN_V25	Yes			
64	✓		HEX5[1]	Location	PIN_AA28	Yes			
65	✓		HEX5[2]	Location	PIN_Y27	Yes			
66	✓		HEX5[3]	Location	PIN_AB27	Yes			
67	✓		HEX5[4]	Location	PIN_AB26	Yes			
68	✓		HEX5[5]	Location	PIN_AA26	Yes			
69	✓		HEX5[6]	Location	PIN_AA25	Yes			
70	✓		HEX5[0]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
71	✓		HEX5[1]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
72	✓		HEX5[2]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
73	✓		HEX5[3]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
74	✓		HEX5[4]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
75	✓		HEX5[5]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
76	✓		HEX5[6]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		
77	✓		Y[3]	Location	PIN_AB12	Yes			
78	✓		Y[2]	Location	PIN_AC12	Yes			
79	✓		Y[1]	Location	PIN_AF9	Yes			
80	✓		Y[0]	Location	PIN_AF10	Yes			
81	✓		X[3]	Location	PIN_AD11	Yes			
82	✓		X[2]	Location	PIN_AD12	Yes			
83	✓		X[1]	Location	PIN_AE11	Yes			
84	✓		X[0]	Location	PIN_AC9	Yes			
85	✓		cin	Location	PIN_AD10	Yes			
86	?		SW[9]	Location	PIN_AE12	Yes			
87	✓		Y[3]	I/O Standard	3.3-V LVTTL	Yes	bcdsum		

Figure 4.9: Pin assignment of part4

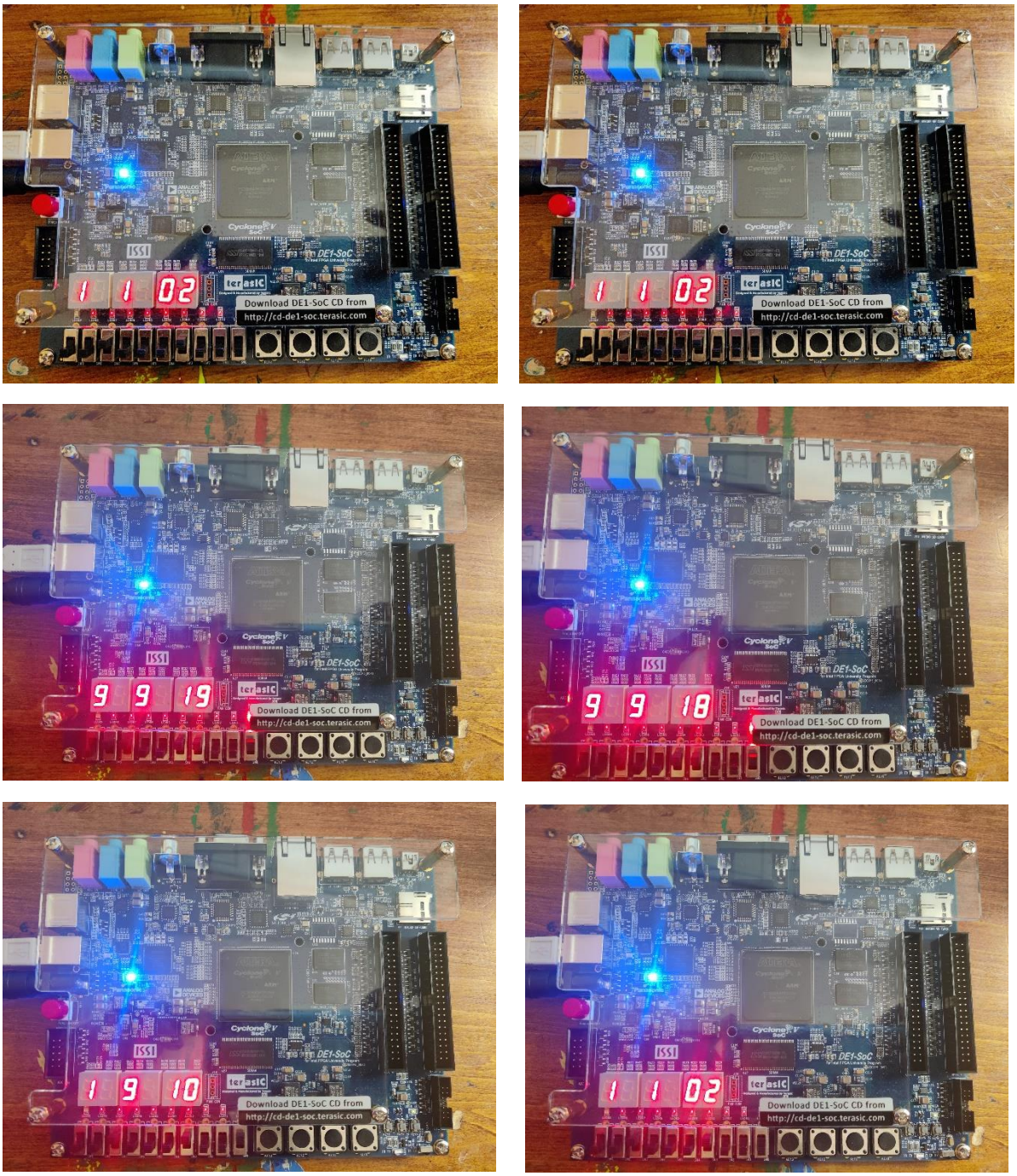


Figure 4.10: Sample output for part4

- **Part5:** The part5 is another approach to solve circuit of part 4. In part4, we are not allowed to use any if-else statement, but part5 helps to know uses of if-else and stuffs and also same output materials and things in part 4. The code design screenshot and a sample output what I get are given below:

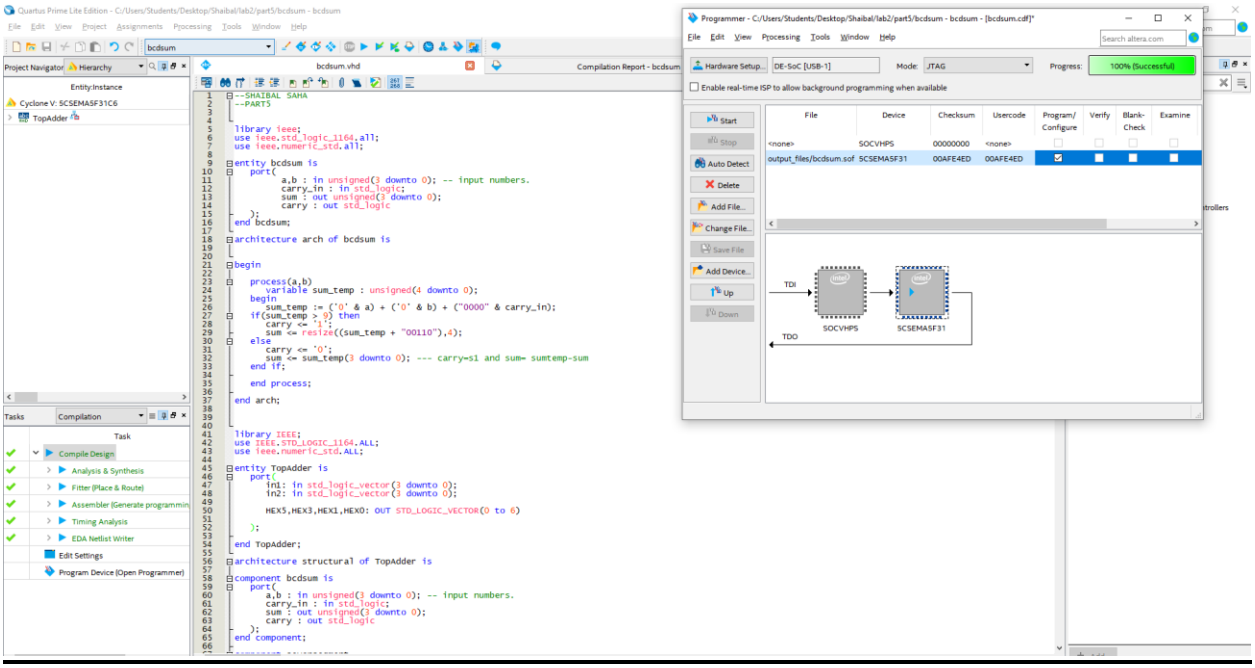


Figure 5.1: code and analysis screenshot for part5

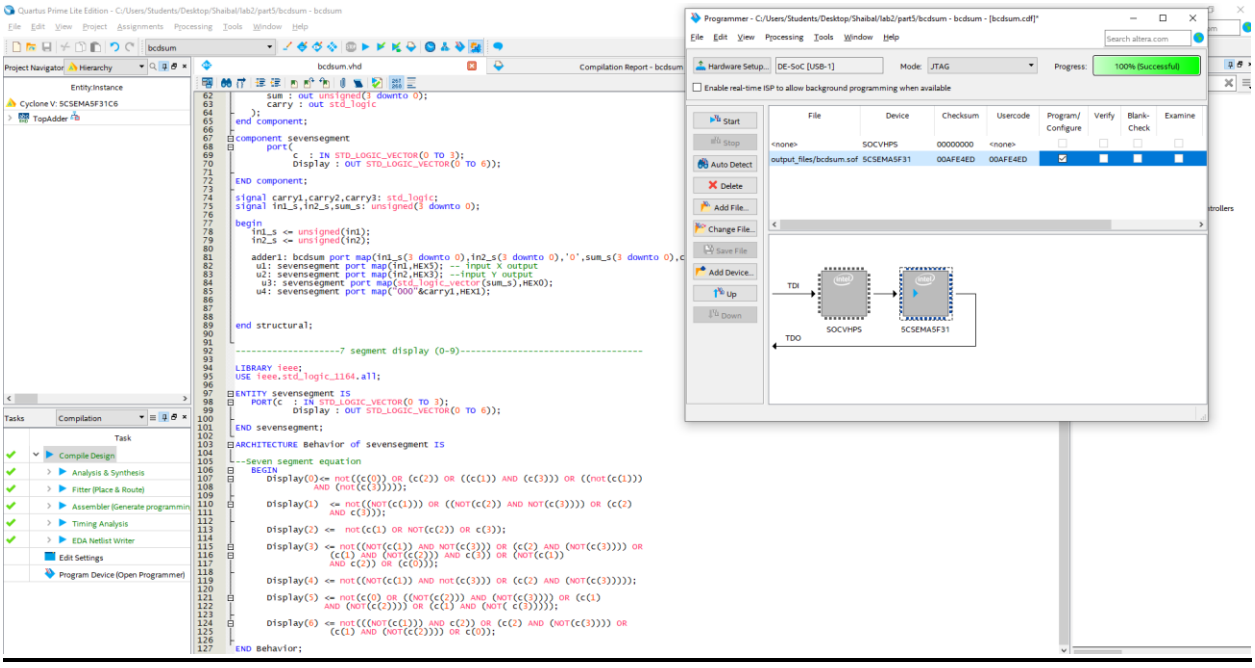


Figure 5.2: code and analysis screenshot for part5

bcdsum.vhd | Compilation Report - bcdsum | Assignment E

<<new>> | Filter on node names: *

	tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
1	✓		HEX0[0]	Location	PIN_AE26	Yes			
2	✓		HEX0[1]	Location	PIN_AE27	Yes			
3	✓		HEX0[2]	Location	PIN_AE28	Yes			
4	✓		HEX0[3]	Location	PIN_AG27	Yes			
5	✓		HEX0[4]	Location	PIN_AF28	Yes			
6	✓		HEX0[5]	Location	PIN_AG28	Yes			
7	✓		HEX0[6]	Location	PIN_AH28	Yes			
8	✓		HEX1[0]	Location	PIN_AJ29	Yes			
9	✓		HEX1[1]	Location	PIN_AH29	Yes			
10	✓		HEX1[2]	Location	PIN_AH30	Yes			
11	✓		HEX1[3]	Location	PIN_AG30	Yes			
12	✓		HEX1[4]	Location	PIN_AF29	Yes			
13	✓		HEX1[5]	Location	PIN_AF30	Yes			
14	✓		HEX1[6]	Location	PIN_AD27	Yes			
15	✓		HEX3[0]	Location	PIN_AD26	Yes			
16	✓		HEX3[1]	Location	PIN_AC27	Yes			
17	✓		HEX3[2]	Location	PIN_AD25	Yes			
18	✓		HEX3[3]	Location	PIN_AC25	Yes			
19	✓		HEX3[4]	Location	PIN_AB28	Yes			
20	✓		HEX3[5]	Location	PIN_AB25	Yes			
21	✓		HEX3[6]	Location	PIN_AB22	Yes			
22	✓		HEX5[0]	Location	PIN_V25	Yes			
23	✓		HEX5[1]	Location	PIN_AA28	Yes			
24	✓		HEX5[2]	Location	PIN_Y27	Yes			
25	✓		HEX5[3]	Location	PIN_AB27	Yes			
26	✓		HEX5[4]	Location	PIN_AB26	Yes			
27	✓		HEX5[5]	Location	PIN_AA26	Yes			
28	✓		HEX5[6]	Location	PIN_AA25	Yes			
29	✓		in2[3]	Location	PIN_AB12	Yes			
30	✓		in2[2]	Location	PIN_AC12	Yes			
31	✓		in2[1]	Location	PIN_AF9	Yes			
32	✓		in2[0]	Location	PIN_AF10	Yes			
33	✓		in1[3]	Location	PIN_AD11	Yes			
34	✓		in1[2]	Location	PIN_AD12	Yes			

Figure 5.3: Pin assignment for part5

bcdsum.vhd | Compilation Report - bcdsum | Assignme

<<new>> | Filter on node names: *

	tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
36	✓		in1[0]	Location	PIN_AC9	Yes			
37	?		carry_in	Location	PIN_AD10	Yes			
38	?		SW[9]	Location	PIN_AE12	Yes			
39	✓		HEX0[0]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
40	✓		HEX0[1]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
41	✓		HEX0[2]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
42	✓		HEX0[3]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
43	✓		HEX0[4]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
44	✓		HEX0[5]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
45	✓		HEX0[6]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
46	✓		HEX1[0]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
47	✓		HEX1[1]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
48	✓		HEX1[2]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
49	✓		HEX1[3]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
50	✓		HEX1[4]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
51	✓		HEX1[5]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
52	✓		HEX1[6]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
53	✓		HEX3[0]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
54	✓		HEX3[1]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
55	✓		HEX3[2]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
56	✓		HEX3[3]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
57	✓		HEX3[4]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
58	✓		HEX3[5]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
59	✓		HEX3[6]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
60	✓		HEX5[0]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
61	✓		HEX5[1]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
62	✓		HEX5[2]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
63	✓		HEX5[3]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
64	✓		HEX5[4]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
65	✓		HEX5[5]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
66	✓		HEX5[6]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
67	?		SW[0]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
68	?		SW[1]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		
69	?		SW[2]	I/O Standard	3.3-V LVTTL	Yes	TopAdder		

Figure 5.4: Pin assignment for part5

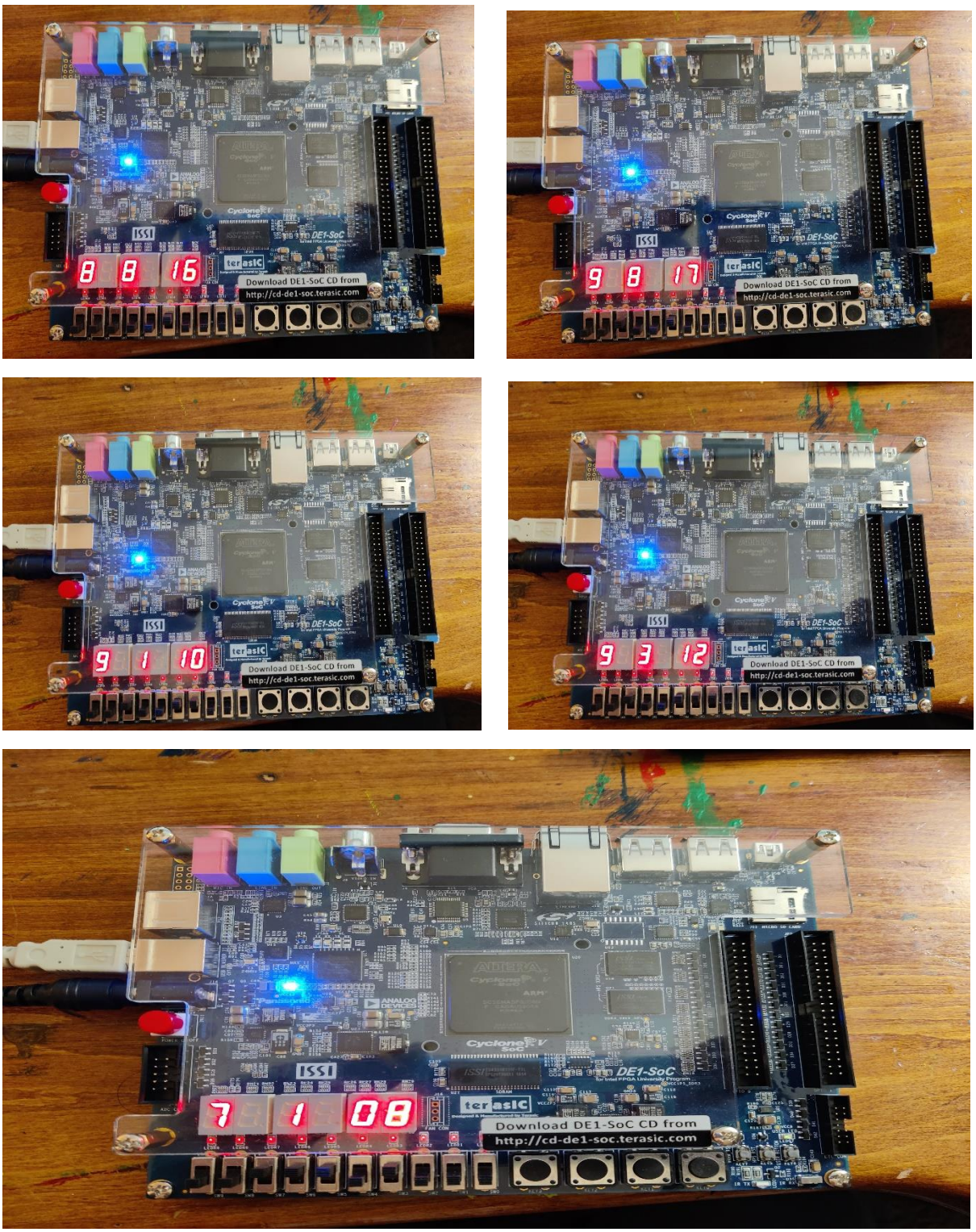


Figure 5.5: Sample output for part5

- **Part6:** The part6 is the combinational design of 6-bit binary number to 2-bit decimal number. We need to redesign comparator for 6 bit and mux and divide out output like under 10, under 20 means multiples of tens. It can produce up to 63 decimal numbers. The complete code design screenshot and a sample output what I get are given below:

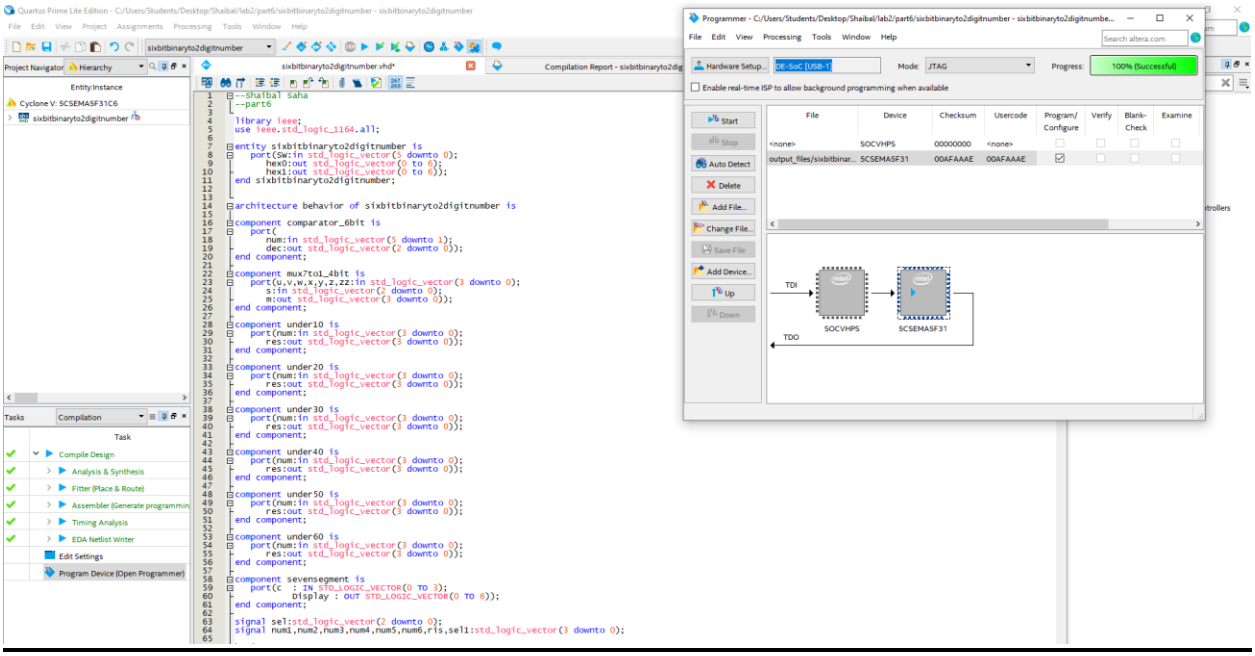


Figure 6.1: code and analysis screenshot for part6

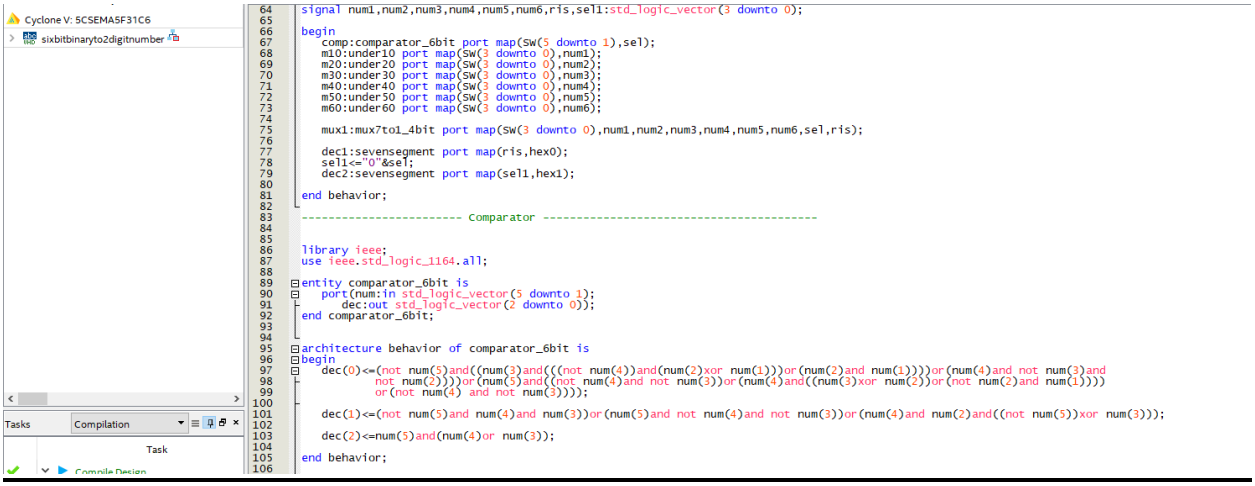


Figure 6.2: code and analysis screenshot for part6(main function output section and comparator)

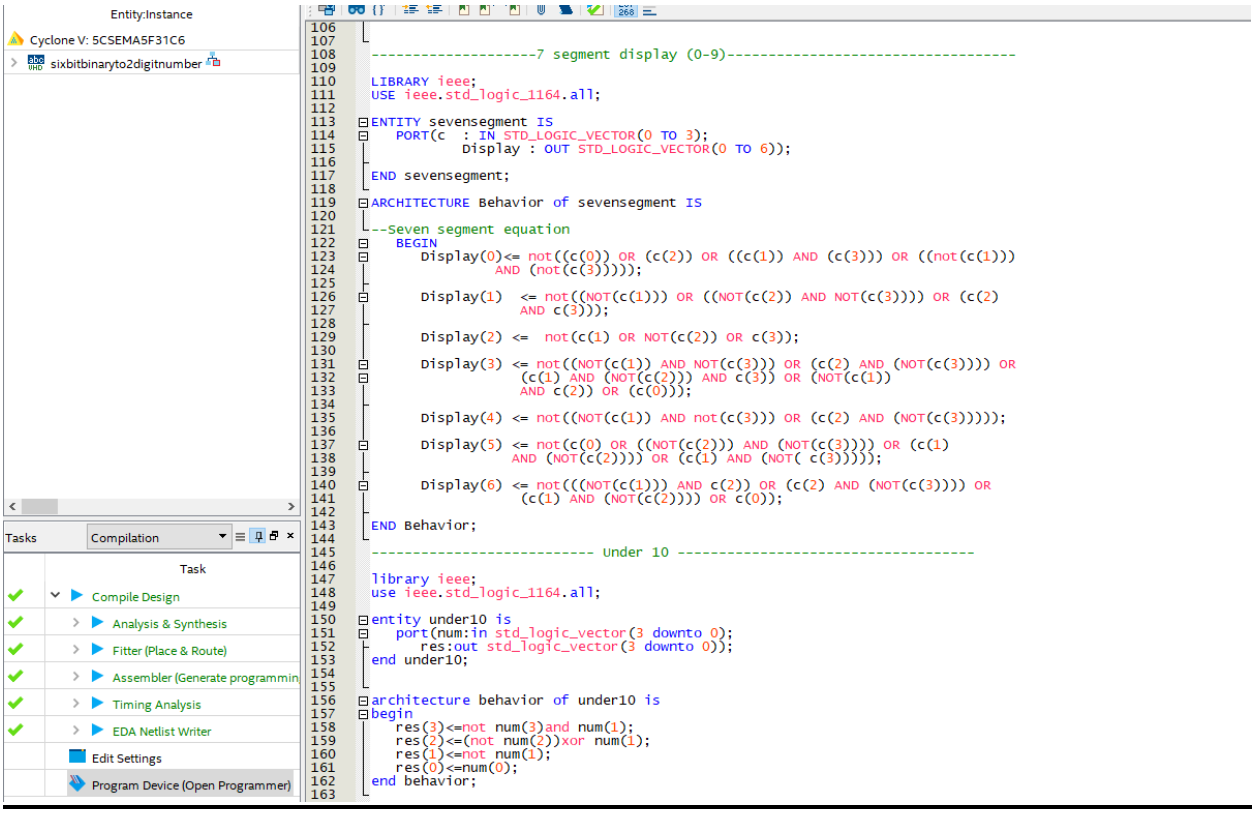


Figure 6.3: code and analysis screenshot for part6(7 segment and equation for under 10 numbers)

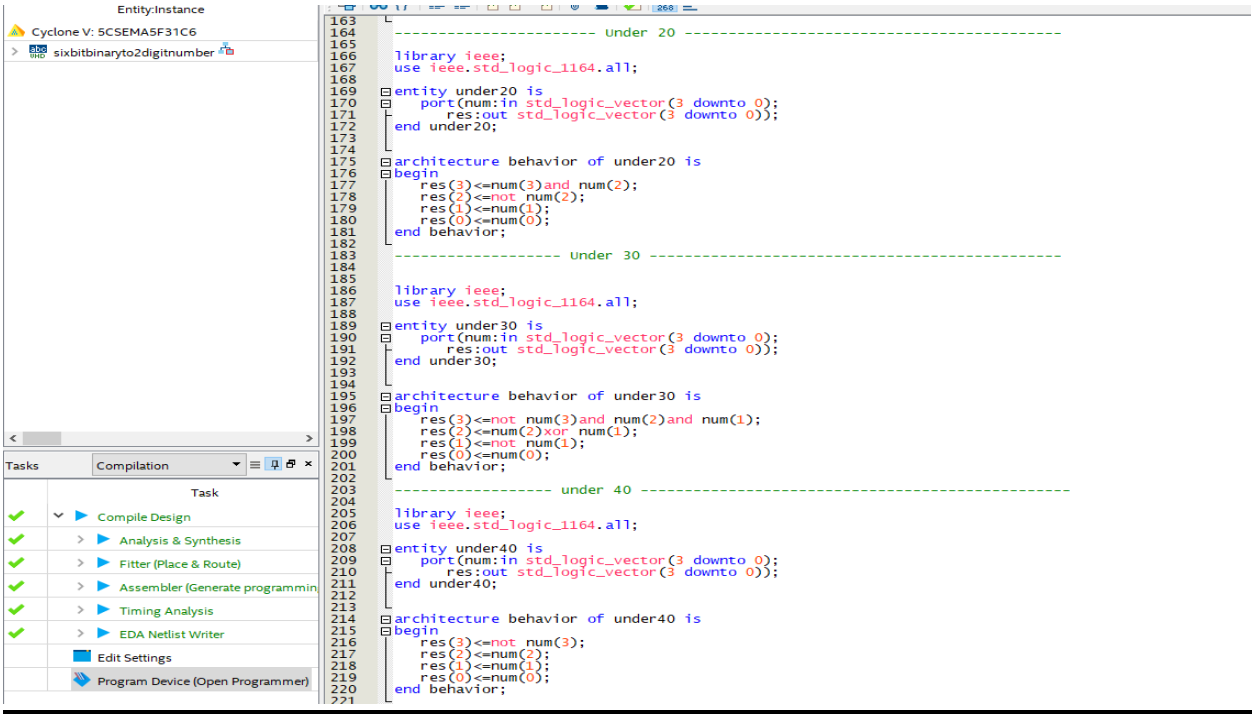


Figure 6.2: code and analysis screenshot for part6(equation for under 20,30,40 numbers)

```
------------------ under 50 ----------------------------------------------------

library ieee;
use ieee.std_logic_1164.all;

entity under50 is
    port(num:in std_logic_vector(3 downto 0);
         res:out std_logic_vector(3 downto 0));
end under50;

architecture behavior of under50 is
begin
    res(3)<=num(3)and num(1);
    res(2)<=(not num(2))xor num(1);
    res(1)<=not num(1);
    res(0)<=num(0);
end behavior;

------------------- under 60 ----------------------------------------------------

library ieee;
use ieee.std_logic_1164.all;

entity under60 is
    port(num:in std_logic_vector(3 downto 0);
         res:out std_logic_vector(3 downto 0));
end under60;

architecture behavior of under60 is
begin
    res(3)<='0';
    res(2)<='0';
    res(1)<=num(1);
    res(0)<=num(0);
end behavior;
 --------------------mux 2 to 1 --------------------------------

library ieee;
use ieee.std_logic_1164.all;

entity Mux2to1 is
    port(x,y,s:in std_logic;
         m:out std_logic);
end Mux2to1;

architecture Behavior of Mux2to1 is
begin
    P1:m<=(not s and x) or (s and y);
end Behavior;
```

Figure 6.3: code and analysis screenshot for part6(equation for 50,60 numbers and 7-segment display equation)

```
 --------------------mux 4o 1 -------------------------------

 library ieee;
use ieee.std_logic_1164.all;

entity mux2to1_4bit is
    port(x,y:in std_logic_vector(3 downto 0);
         s:in std_logic;
         m:out std_logic_vector(3 downto 0));
end Mux2to1_4bit;

architecture behavior of Mux2to1_4bit is

component Mux2to1 is
    port(x,y,s:in std_logic;
         m:out std_logic);
end component;

begin
    mux1:Mux2to1 port map(x(0),y(0),s,m(0));
    mux2:Mux2to1 port map(x(1),y(1),s,m(1));
    mux3:Mux2to1 port map(x(2),y(2),s,m(2));
    mux4:Mux2to1 port map(x(3),y(3),s,m(3));
end behavior;

------------------ mux ----------------------------------------

library ieee;
use ieee.std_logic_1164.all;

entity mux7to1_4bit is
    port(u,v,w,x,y,z,zz:in std_logic_vector(3 downto 0);
         s:in std_logic_vector(2 downto 0);
         m:out std_logic_vector(3 downto 0));
end Mux7to1_4bit;

architecture behavior of Mux7to1_4bit is

component mux2to1_4bit is
    port(x,y:in std_logic_vector(3 downto 0);
         s:in std_logic;
         m:out std_logic_vector(3 downto 0));
end component;

signal s1,s2,s3,s4,s5:std_logic_vector(3 downto 0);

begin
    mux1:mux2to1_4bit port map(u,v,s(0),s1);
    mux2:mux2to1_4bit port map(w,x,s(0),s2);
    mux3:mux2to1_4bit port map(y,z,s(0),s3);
    mux4:mux2to1_4bit port map(s1,s2,s(1),s4);
    mux5:mux2to1_4bit port map(s3,zz,s(1),s5);
    mux6:mux2to1_4bit port map(s4,s5,s(2),m);
end behavior;
```

Figure 6.4: code and analysis screenshot for part6(designing mux using 4 to 1 and 2 to 1 mux)

sixbitbinaryto2digitnumber.vhd* | Compilation Report - sixbitbinaryto2digitnumber | Assignm

<<new>> | Filter on node names: *

	tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
1	✔		HEX0[0]	Location	PIN_AE26	Yes			
2	✔		HEX0[1]	Location	PIN_AE27	Yes			
3	✔		HEX0[2]	Location	PIN_AE28	Yes			
4	✔		HEX0[3]	Location	PIN_AG27	Yes			
5	✔		HEX0[4]	Location	PIN_AF28	Yes			
6	✔		HEX0[5]	Location	PIN_AG28	Yes			
7	✔		HEX0[6]	Location	PIN_AH28	Yes			
8	✔		HEX1[0]	Location	PIN_AJ29	Yes			
9	✔		HEX1[1]	Location	PIN_AH29	Yes			
10	✔		HEX1[2]	Location	PIN_AH30	Yes			
11	✔		HEX1[3]	Location	PIN_AG30	Yes			
12	✔		HEX1[4]	Location	PIN_AF29	Yes			
13	✔		HEX1[5]	Location	PIN_AF30	Yes			
14	✔		HEX1[6]	Location	PIN_AD27	Yes			
15	✔		SW[0]	Location	PIN_AB12	Yes			
16	✔		SW[1]	Location	PIN_AC12	Yes			
17	✔		SW[2]	Location	PIN_AF9	Yes			
18	✔		SW[3]	Location	PIN_AF10	Yes			
19	✔		SW[4]	Location	PIN_AD11	Yes			
20	✔		SW[5]	Location	PIN_AD12	Yes			
21	?		SW[6]	Location	PIN_AE11	Yes			
22	?		SW[7]	Location	PIN_AC9	Yes			
23	?		SW[8]	Location	PIN_AD10	Yes			
24	?		SW[9]	Location	PIN_AE12	Yes			
25	✔		SW[0]	I/O Standard	3.3-V LVTTL	Yes	sixbitb...tnumber		
26	✔		SW[1]	I/O Standard	3.3-V LVTTL	Yes	sixbitb...tnumber		
27	✔		SW[2]	I/O Standard	3.3-V LVTTL	Yes	sixbitb...tnumber		
28	✔		SW[3]	I/O Standard	3.3-V LVTTL	Yes	sixbitb...tnumber		
29	✔		SW[4]	I/O Standard	3.3-V LVTTL	Yes	sixbitb...tnumber		
30	✔		SW[5]	I/O Standard	3.3-V LVTTL	Yes	sixbitb...tnumber		
31	?		SW[6]	I/O Standard	3.3-V LVTTL	Yes	sixbitb...tnumber		
32	?		SW[7]	I/O Standard	3.3-V LVTTL	Yes	sixbitb...tnumber		
33	?		SW[8]	I/O Standard	3.3-V LVTTL	Yes	sixbitb...tnumber		
34	?		SW[9]	I/O Standard	3.3-V LVTTL	Yes	sixbitb...tnumber		

Figure 6.4: Pin assignment for part 6

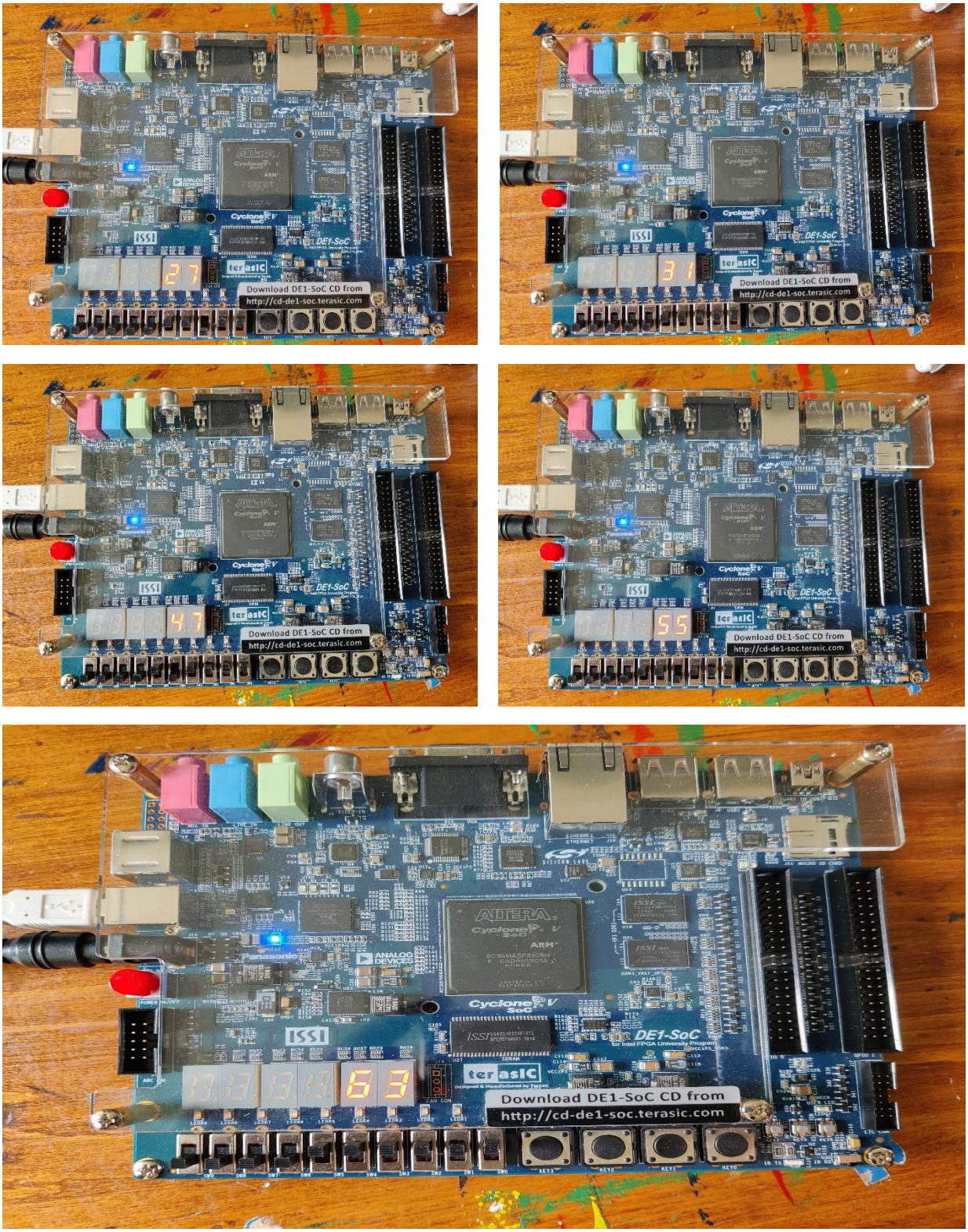


Figure 5.5: Sample output for part6

Note: All the source codes are inside lab folder.